⚡ L23: Advanced Inheritance

Commander Override Protocol

Neon City Cyber Academy

Module 5: Object-Oriented

🎯 New Concepts Today

1. Method Overriding

Replace parent's method with child's version

2. super()

Call parent's method from child

3. Polymorphism

Same method name, different behavior

Master these → OOP mastery complete!

🔄 Method Overriding

Override = Replace parent's method in child

```
class Droid:
    def greet(self):
        print(f"Greetings. I am {self.name}, Worker Unit.")

class CommanderDroid(Droid):
    def greet(self):  # OVERRIDES parent version
        print(f"ATTENTION. Commander {self.name} taking control.")
```

Child's version replaces parent's!

💡 When to Override

Override when:

- Child needs DIFFERENT behavior
- Parent's version isn't suitable
- Need specialized functionality

Example:

- Worker greets politely
- Commander greets with authority
- SAME action, DIFFERENT implementation

🔧 Using super()

```
class Droid:
    def __init__(self, name, battery_level):
        self.name = name
        self.battery_level = battery_level

class CommanderDroid(Droid):
    def __init__(self, name, battery_level, rank):
        super().__init__(name, battery_level)  # Call parent init
        self.rank = rank  # Add new attribute
```

super() = Call the parent class version

🎯 Why Use super()?

Without super():

```
def __init__(self, name, battery_level, rank):
    self.name = name  # Duplicate code!
    self.battery_level = battery_level
    self.rank = rank
```

With super():

```
def __init__(self, name, battery_level, rank):
    super().__init__(name, battery_level)  # Reuse!
    self.rank = rank
```

Avoids code duplication!

Complete Example

```
class Droid:
    def __init__(self, name, battery_level):
        self.name = name
        self.battery_level = battery_level

    def greet(self):
        print(f"Greetings. I am {self.name}, Worker Unit.")

class CommanderDroid(Droid):
    def __init__(self, name, battery_level, rank):
        super().__init__(name, battery_level)
        self.rank = rank

    def greet(self):
        print(f"Commander {self.name}, Rank {self.rank}, reporting.")
```

🎮 Polymorphism in Action

Polymorphism = Same interface, different implementations

```
fleet = [
    Droid('WORKER-1', 90),
    Droid('WORKER-2', 85),
    CommanderDroid('CHIEF-X', 100, 'Captain')
]

for unit in fleet:
    unit.greet()  # Python knows which version to call!
```

Output:

💬 Understanding Polymorphism

> "Same method name, different behavior based on object type"

- Workers use Droid's `greet()`
- Commanders use CommanderDroid's `greet()`
- Python automatically picks the right version!

This is OOP magic!

Inheritance Levels

```
Droid
├── __init__()
├── greet()
│
├── SecurityDroid
│   ├── __init__() [inherited]
│   ├── greet() [inherited]
│   └── patrol() [new]
│
└── CommanderDroid
    ├── __init__() [overridden with super()]
    ├── greet() [overridden]
    └── rank [new attribute]
```

🎯 Method Resolution Order

When calling `obj.method()`, Python checks:

1. Child class first
2. Parent class if not found
3. Grandparent if exists
4. Continue up the chain

First match wins!

🔧 Advanced super() Usage

```
class CommanderDroid(Droid):
    def greet(self):
        super().greet()  # Call parent's greet first
        print(f"My rank is {self.rank}.")

# Output:
# Greetings. I am CHIEF-X, Worker Unit.
# My rank is Captain.
```

Extend behavior instead of replacing!

💡 Override vs Extend

Override (Replace):

```
def greet(self):
    print("New message")  # Completely new
```

Extend (Add to):

```
def greet(self):
    super().greet()  # Use parent's first
    print("Additional message")  # Then add
```

🎯 Factory Inspection Demo

```
# Mixed fleet
fleet = [Droid('W1', 100), CommanderDroid('CMD', 100, 'General')]

print("=== FACTORY ROLL CALL ===")
for unit in fleet:
    unit.greet()
    if hasattr(unit, 'rank'):  # Check if has rank
        print(f"  → Rank: {unit.rank}")
```

hasattr() checks if attribute exists!

✅ Full OOP Mastery!

You Completed:

- ✅ Objects and Methods (L18)
- ✅ Events (L19)
- ✅ Paint Project (L20)
- ✅ Classes (L21)
- ✅ Inheritance (L22)
- ✅ Method Overriding & super() (L23)

You are now an OOP developer!

🎓 What You Can Build

With OOP Skills:

- 🎮 Video games (characters, items, enemies)
- 🤖 Simulations (robots, ecosystems)
- 📊 Data models (users, products, orders)
- 🚗 Vehicle systems (cars, planes, boats)

Everything in modern software uses OOP!

🏆 Final Challenge

Build a Complete Droid Factory:

1. Create `Droid` base class
2. Add 3 child classes with unique methods
3. Override `greet()` in each child
4. Use `super()` for extended attributes
5. Create a list of mixed droids
6. Loop and demonstrate polymorphism

Show everything you learned!

Graduation Message

Congratulations, Agent!

You've completed Module 5: Object-Oriented Programming.

You can now:

- Design class hierarchies
- Build reusable code
- Create complex systems

Next mission awaits. Keep coding! 🚀